

CSC3066

Deep Learning

Training Perceptron with Gradient Descent
and Backpropagation

Anna Jurek-Loughrey

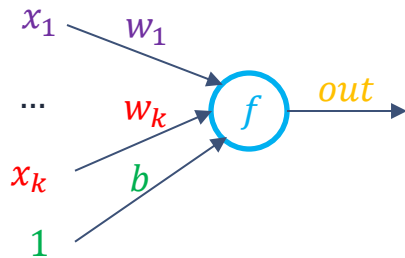


What's to come today

- ❑ Recap on Perceptron
- ❑ Training Perceptron with Gradient Descent and Backpropagation

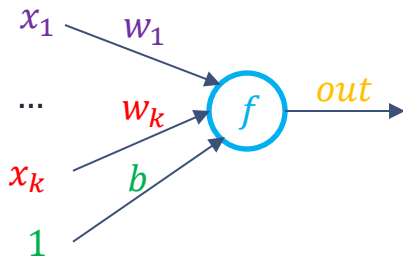


Backpropagation: Perceptron





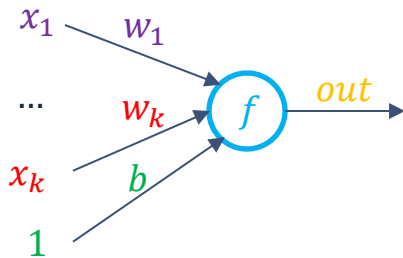
Backpropagation: Perceptron



$$out = f(\overbrace{x_1 w_1 + \dots + x_k w_k + b}^{in})$$



Backpropagation: Perceptron

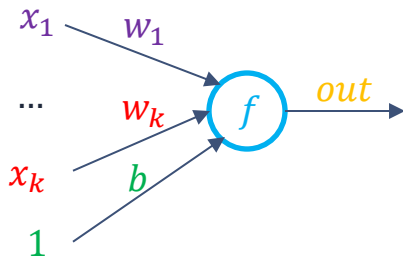


$$out = f(\overbrace{x_1 w_1 + \dots + x_k w_k + b}^{in})$$

$$Error = \frac{1}{2} (target_{output} - out)^2$$



Backpropagation: Perceptron



$$out = f(\overbrace{x_1 w_1 + \dots + x_k w_k + b}^{in})$$

$$Error = \frac{1}{2} (target_{output} - out)^2 = \frac{1}{2} \left(target_{output} - \underbrace{f(x_1 w_1 + \dots + x_k w_k + b)}_{\text{Composite function}} \right)^2$$



Backpropagation: Perceptron

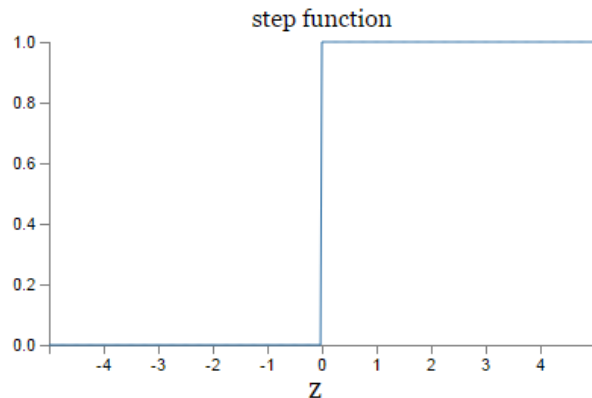
- ❑ The error/loss function depends (indirectly) on variables $w_1, \dots, w_k, b$.
- ❑ If we can calculate $\frac{\partial E}{\partial w_1}, \dots, \frac{\partial E}{\partial w_k}, \frac{\partial E}{\partial b}$ then we will know how to update (decrease or increase) value of each of the parameters so that the error decreases.
- ❑ This depends on the error surface being smooth (i.e. differentiable with respect to the parameters).
- ❑ Crucially, this has implications for both how we calculate the error, and the activation function we use to calculate neuron output.



Backpropagation: Perceptron

❑ Step function problem:

- Non-differentiable at $x = 0$ and it has **0** derivative elsewhere. This means that gradient descent won't be able to make a progress in updating the weights.
- A small change in the weights of any single neuron in the network can cause the output of that neuron to completely flip.

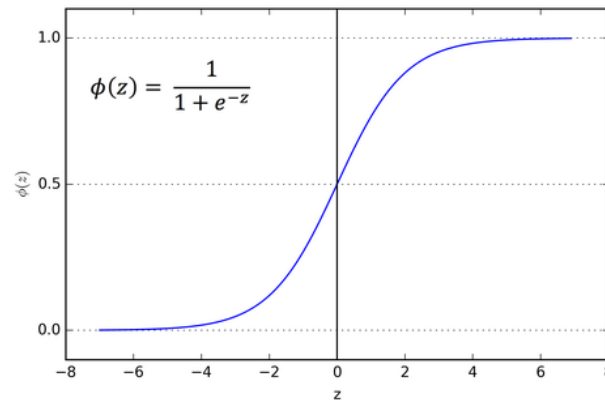




Backpropagation: Perceptron

□ Solution: a *sigmoid* neuron

- Similar to step function but modified so that small changes in their weights and bias cause only a small change in their output.
- A “smoothed out” perceptron



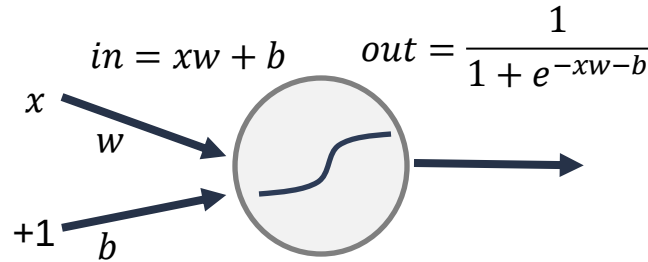
$$out = \sigma(in) = \frac{1}{1 + e^{-in}}$$



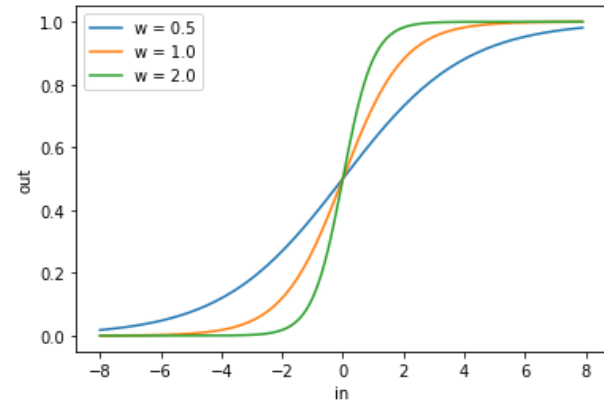
Backpropagation: Perceptron

□ Sigmoid neuron

- ▶ Let's consider a single neuron with one input and one output



- ▶ Changing the weights we can change the slope of the output of the sigmoid activation function

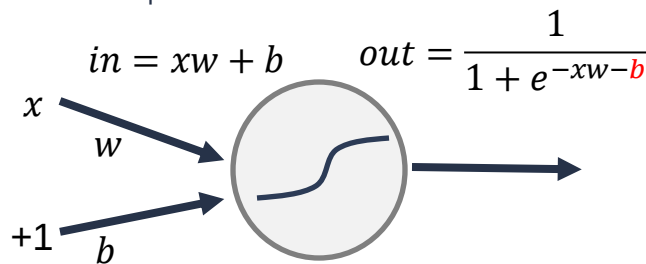




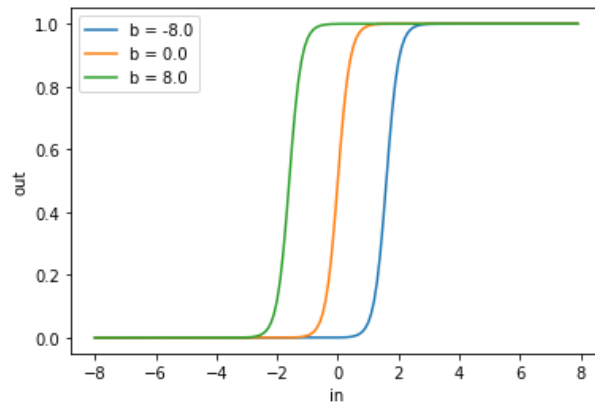
Backpropagation: Perceptron

□ Sigmoid neuron

- ▶ Let's consider a single neuron with one input and one output



- ▶ By changing the bias term, we can offset the sigmoid activation function.

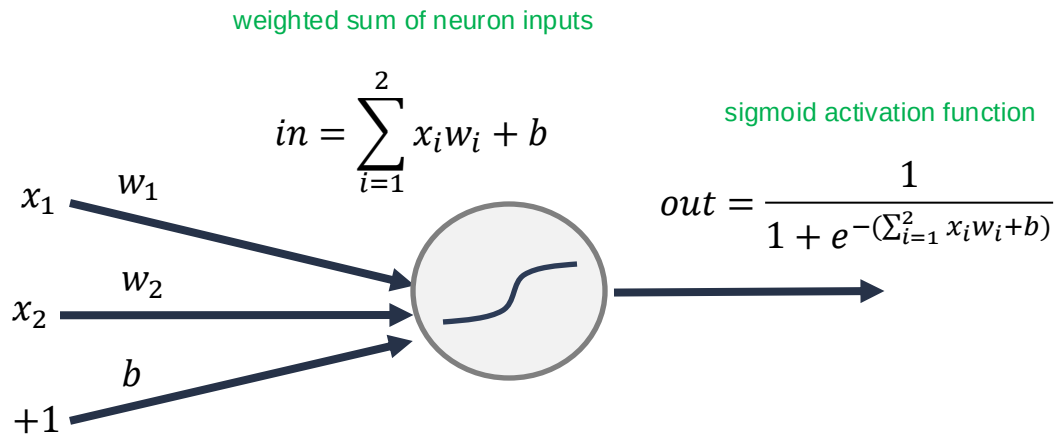




Backpropagation: Perceptron

- Let's consider a simple network with two inputs and a single neuron with sigmoid activation function:

Input:



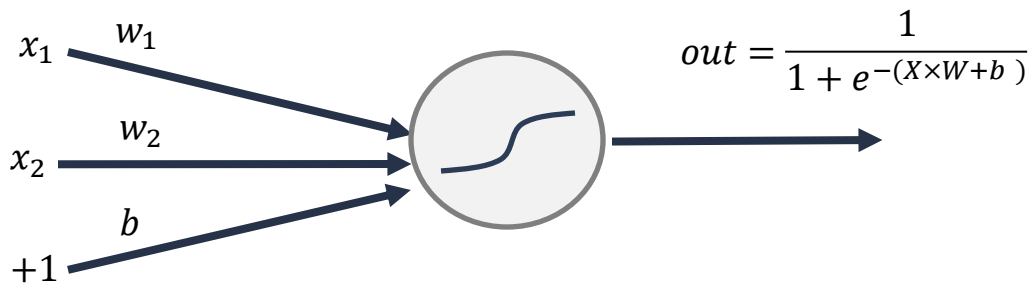


Backpropagation: Perceptron

□ Matrix notation:

$$X = [x_1, x_2] \quad W = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \quad b\text{-scalar}$$

$$in = X \times W + b$$

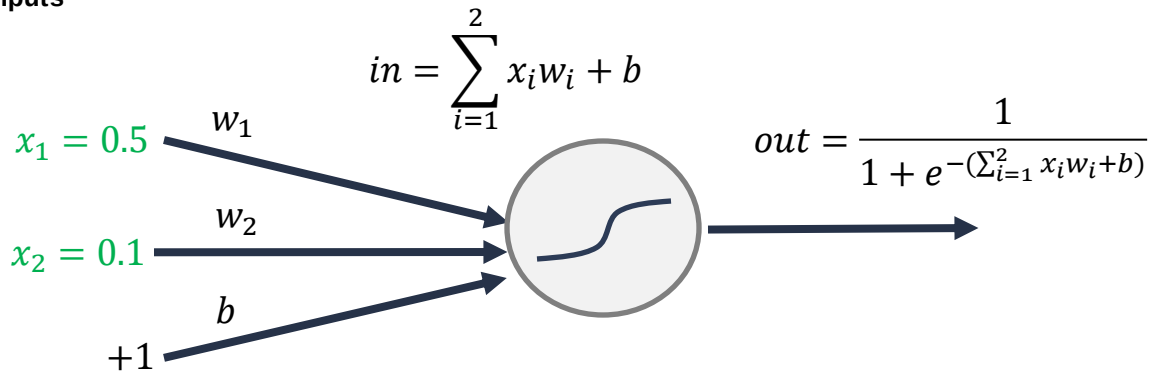




Backpropagation: Perceptron

- Let's consider a single training example: given inputs **0.5** and **0.1**, we want the network to output **0.9**.

inputs

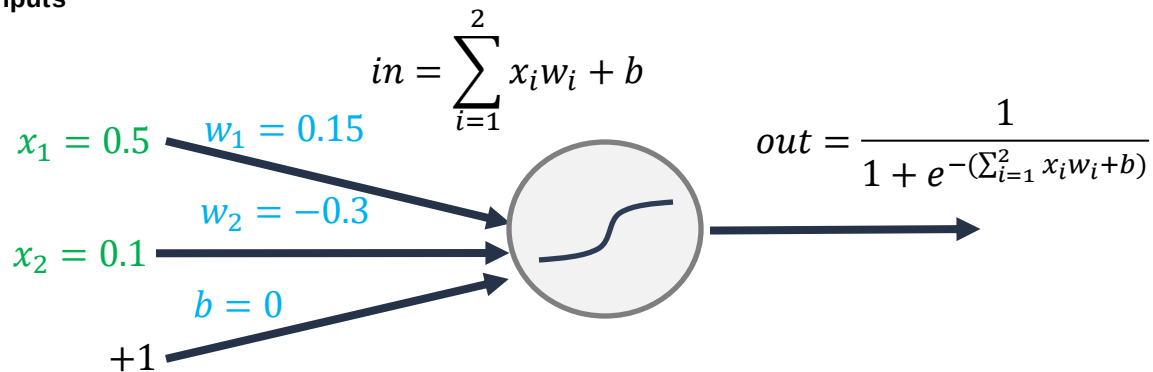




Backpropagation: Perceptron

- ☐ Let's initialise the weights and the bias with random values.
- ☐ We can sample from Uniform or Normal distribution, for example.
- ☐ We could also set all the weights to zero.
- ☐ We will cover more on weights initialisation later in the course.

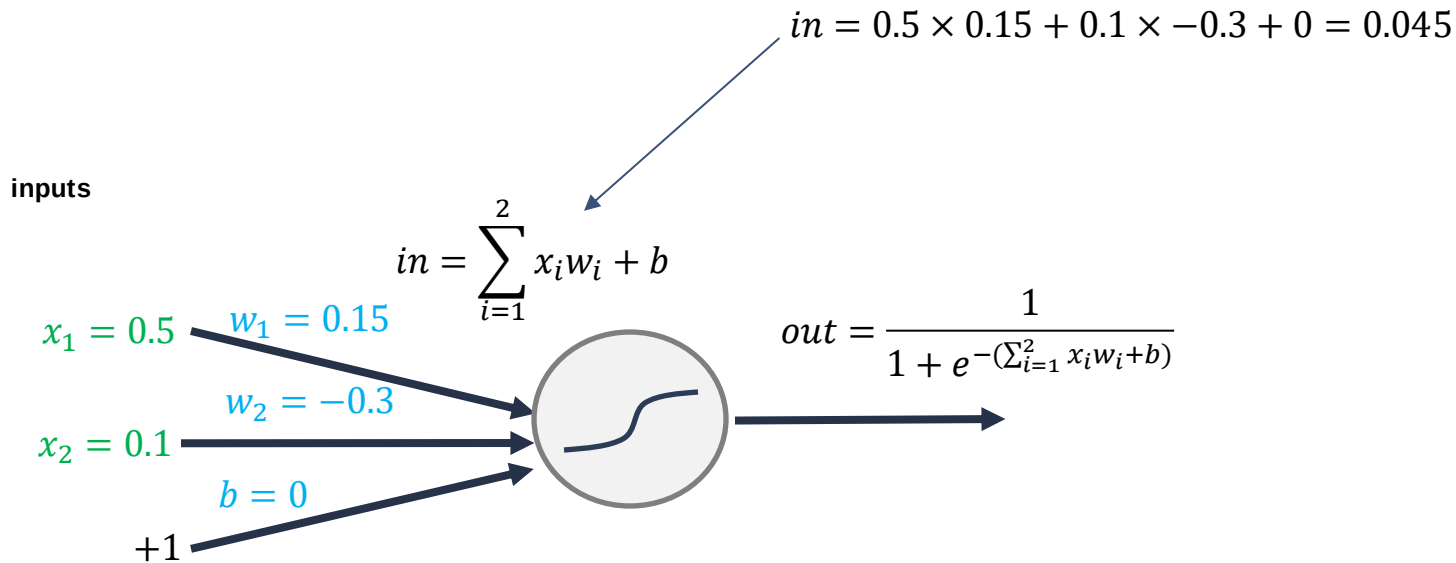
inputs





Backpropagation: Perceptron

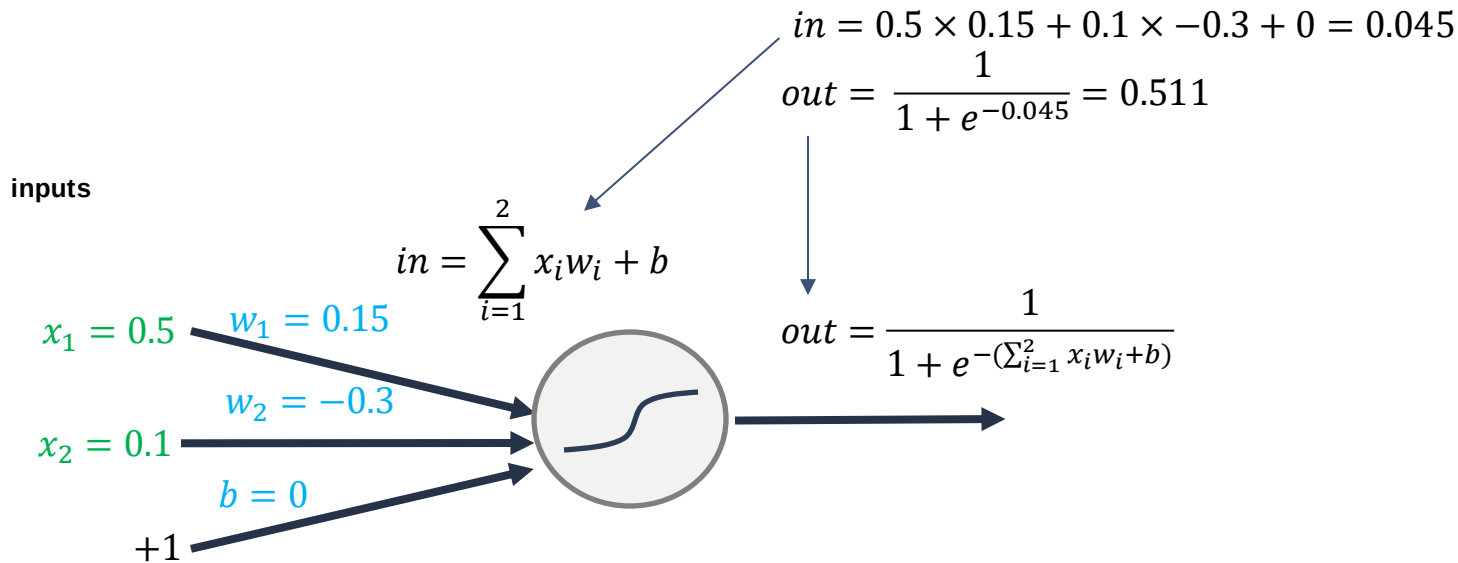
□ Let's see what the network currently predicts:





Backpropagation: Perceptron

□ Let's see what the network currently predicts:





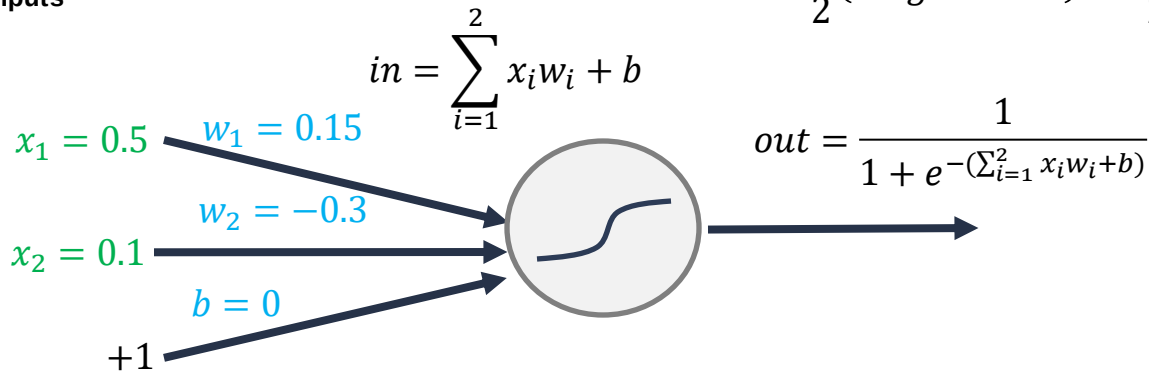
Backpropagation: Perceptron

□ Now we can calculate the error:

$$out = \frac{1}{1 + e^{-0.045}} = 0.511$$

$$E = \frac{1}{2} (target - out)^2 = \frac{1}{2} (0.9 - 0.511)^2 = 0.0757$$

inputs





Backpropagation: Perceptron

- ❑ Now we need to update each of the weights in the network so that they cause the actual output to be closer to the target output (i.e. minimise the error)
- ❑ We want to know how much a change in each weight and the bias affects the total error:

$$\frac{\partial E}{\partial w_i} = ? \quad \frac{\partial E}{\partial b} = ?$$



Backpropagation: Perceptron

$$E = \frac{1}{2} \left(t - \frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2 + b)}} \right)^2$$



Backpropagation: Perceptron

$$E = \frac{1}{2} \left(t - \frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2 + b)}} \right)^2$$

$$in = x_1 w_1 + x_2 w_2 + b \quad out = \frac{1}{1 + e^{-(in)}} \quad E = \frac{1}{2} (t - out)^2$$



Backpropagation: Perceptron

$$E = \frac{1}{2} \left(t - \frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2 + b)}} \right)^2$$

$$in = x_1 w_1 + x_2 w_2 + b \quad out = \frac{1}{1 + e^{-(in)}} \quad E = \frac{1}{2} (t - out)^2$$

$$E = \frac{1}{2} (t - out)^2 = \frac{1}{2} \left(t - \frac{1}{1 + e^{-(in)}} \right)^2 = \frac{1}{2} \left(t - \frac{1}{1 + e^{-(x_1 w_1 + x_2 w_2 + b)}} \right)^2$$



Backpropagation: Perceptron

$$E = \frac{1}{2}(t - \text{out})^2 = \frac{1}{2}\left(t - \frac{1}{1 + e^{-(\text{in})}}\right)^2 = \frac{1}{2}\left(t - \frac{1}{1 + e^{-(x_1w_1 + x_2w_2 + b)}}\right)^2$$

$$\text{in} = x_1w_1 + x_2w_2 + b \quad \text{out} = \frac{1}{1 + e^{-(\text{in})}} \quad E = \frac{1}{2}(t - \text{out})^2$$

$$\frac{\partial E}{\partial w_i}$$



Backpropagation: Perceptron

$$E = \frac{1}{2}(t - \text{out})^2 = \frac{1}{2}\left(t - \frac{1}{1 + e^{-(\text{in})}}\right)^2 = \frac{1}{2}\left(t - \frac{1}{1 + e^{-(x_1w_1 + x_2w_2 + b)}}\right)^2$$

$$\text{in} = x_1w_1 + x_2w_2 + b$$

$$\text{out} = \frac{1}{1 + e^{-(\text{in})}}$$

$$E = \frac{1}{2}(t - \text{out})^2$$

$$\frac{\partial E}{\partial w_i} = \underbrace{\frac{\partial E}{\partial \text{out}} \frac{\partial \text{out}}{\partial \text{in}}}_{\frac{\partial E}{\partial \text{in}}} \frac{\partial \text{in}}{\partial w_i}$$

chain rule



Backpropagation: Perceptron

$$E = \frac{1}{2}(t - \text{out})^2 = \frac{1}{2}\left(t - \frac{1}{1 + e^{-(\text{in})}}\right)^2 = \frac{1}{2}\left(t - \frac{1}{1 + e^{-(x_1w_1 + x_2w_2 + b)}}\right)^2$$

$$\text{in} = x_1w_1 + x_2w_2 + b$$

$$\text{out} = \frac{1}{1 + e^{-(\text{in})}}$$

$$E = \frac{1}{2}(t - \text{out})^2$$

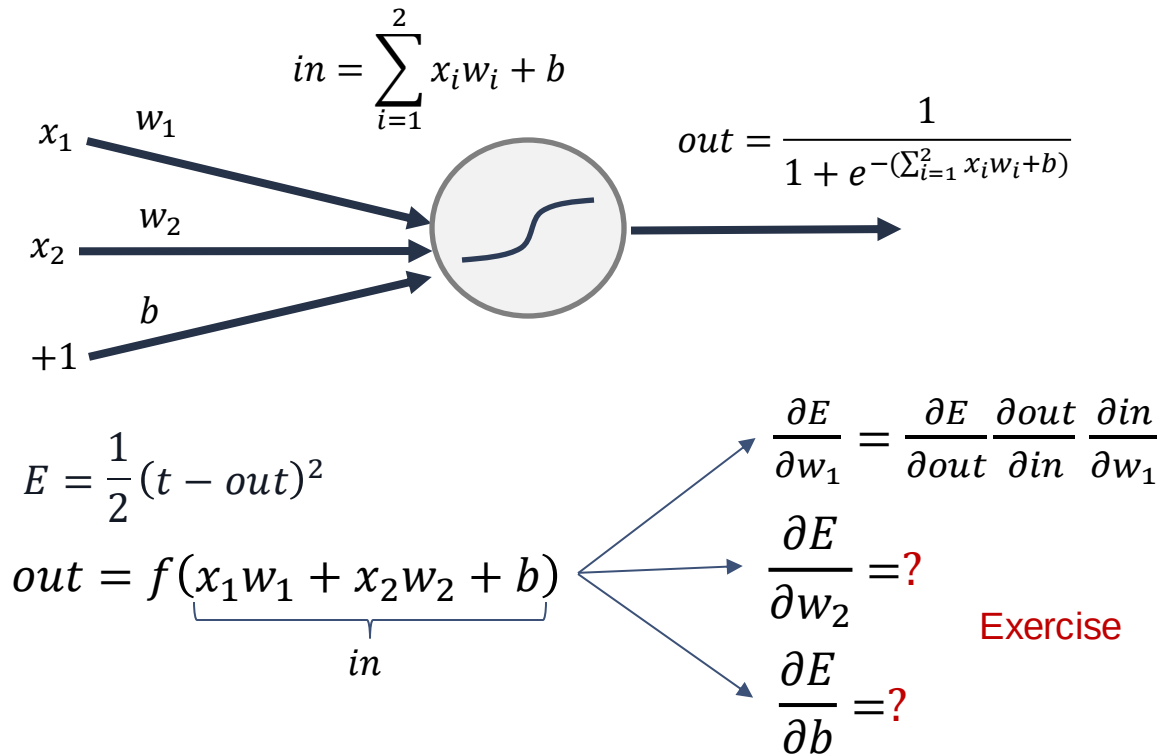
$$\frac{\partial E}{\partial w_i} = \underbrace{\frac{\partial E}{\partial \text{out}} \frac{\partial \text{out}}{\partial \text{in}}}_{\frac{\partial E}{\partial \text{in}}} \frac{\partial \text{in}}{\partial w_i}$$

chain rule

← We need to calculate these three partial derivatives

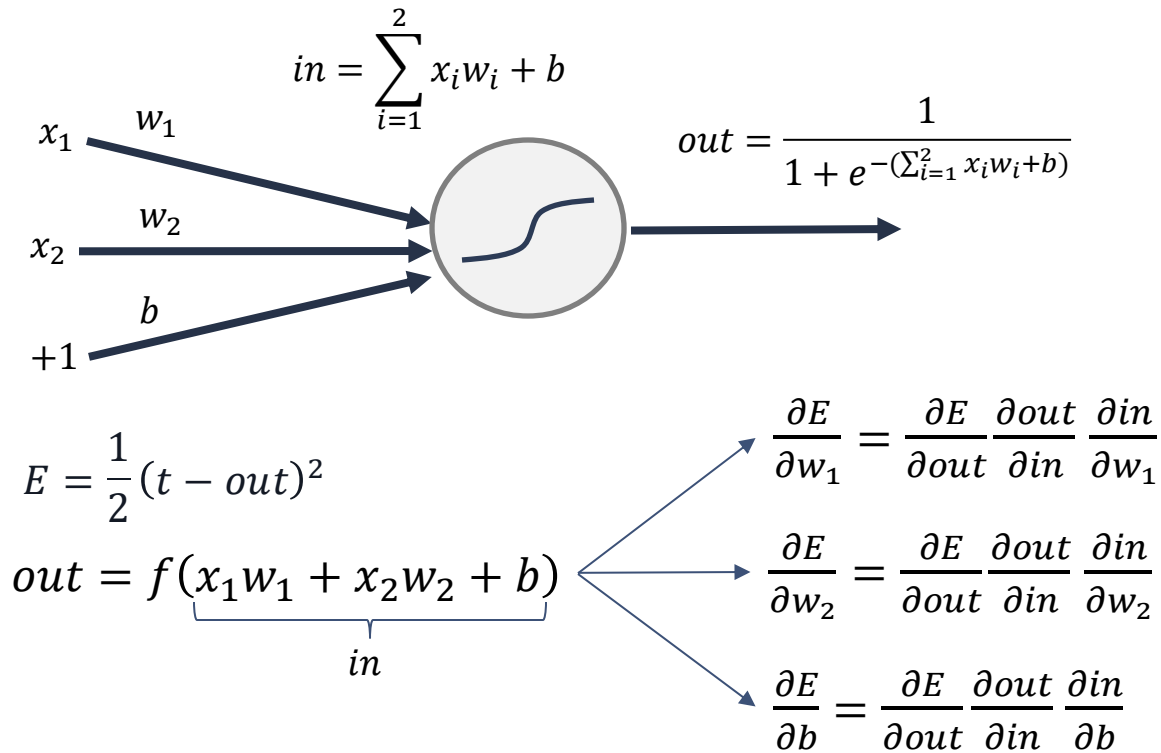


Backpropagation: Perceptron





Backpropagation: Perceptron



Backpropagation: Perceptron

- How do we calculate the partial derivatives?

Backpropagation: Perceptron

- How do we calculate the partial derivatives?

- $E = \frac{1}{2}(t - out)^2$ $\frac{\partial E}{\partial out} = ?$

Backpropagation: Perceptron

- How do we calculate the partial derivatives?

- $E = \frac{1}{2}(t - out)^2$ $\frac{\partial E}{\partial out} = out - t$

Backpropagation: Perceptron

□ How do we calculate the partial derivatives?

- $E = \frac{1}{2}(t - out)^2$ $\frac{\partial E}{\partial out} = out - t$

- $out(in) = \frac{1}{1+e^{-in}}$ $\frac{\partial out}{\partial in} = \frac{1}{1+e^{-in}} \left(1 - \frac{1}{1+e^{-in}} \right)$

Backpropagation: Perceptron

□ How do we calculate the partial derivatives?

- $E = \frac{1}{2}(t - out)^2$ $\frac{\partial E}{\partial out} = out - t$

- $out(in) = \frac{1}{1+e^{-in}}$ $\frac{\partial out}{\partial in} = \frac{1}{1+e^{-in}} \left(1 - \frac{1}{1+e^{-in}}\right)$

- $in(w) = \sum_{i=1}^k x_i w_i + b$ $\frac{\partial in}{\partial w_i} = ?$ $\frac{\partial in}{\partial b} = ?$

Backpropagation: Perceptron

□ How do we calculate the partial derivatives?

- $E = \frac{1}{2}(t - out)^2$ $\frac{\partial E}{\partial out} = out - t$

- $out(in) = \frac{1}{1+e^{-in}}$ $\frac{\partial out}{\partial in} = \frac{1}{1+e^{-in}} \left(1 - \frac{1}{1+e^{-in}}\right)$

- $in(w) = \sum_{i=1}^k x_i w_i + b$ $\frac{\partial in}{\partial w_i} = x_i$ $\frac{\partial in}{\partial b} = 1$



Backpropagation: Perceptron

$$\frac{\partial E}{\partial out} = out - t = 0.511 - 0.9 = -0.389$$

$$\frac{\partial out}{\partial in} = \frac{1}{1 + e^{-in}} \left(1 - \frac{1}{1 + e^{-in}} \right) = 0.511 \times (1 - 0.511) = 0.245$$

$$\frac{\partial in}{\partial w_1} = x_1 = 0.5$$

$$\frac{\partial E}{\partial w_1} = \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} \frac{\partial in}{\partial w_1} = -0.389 \times 0.245 \times 0.5 = -0.049$$



Backpropagation: Perceptron

$$\frac{\partial E}{\partial out} = out - t = 0.511 - 0.9 = -0.389$$

$$\frac{\partial out}{\partial in} = \frac{1}{1 + e^{-in}} \left(1 - \frac{1}{1 + e^{-in}} \right) = 0.511 \times (1 - 0.511) = 0.245$$

$$\frac{\partial in}{\partial w_2} = x_2 = 0.1$$

$$\frac{\partial E}{\partial w_2} = \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} \frac{\partial in}{\partial w_2} = -0.389 \times 0.245 \times 0.1 = -0.01$$



Backpropagation: Perceptron

$$\frac{\partial E}{\partial out} = out - t = 0.511 - 0.9 = -0.389$$

$$\frac{\partial out}{\partial in} = \frac{1}{1 + e^{-in}} \left(1 - \frac{1}{1 + e^{-in}} \right) = 0.511 \times (1 - 0.511) = 0.245$$

$$\frac{\partial in}{\partial b} = 1$$

$$\frac{\partial E}{\partial b} = \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} \frac{\partial in}{\partial b} = -0.389 \times 0.245 \times 1 = -0.098$$



Backpropagation: Perceptron

Putting it all together:

$$\frac{\partial E}{\partial w_1} = -0.049$$

$$\frac{\partial E}{\partial w_2} = -0.01$$

$$\frac{\partial E}{\partial b} = -0.098$$



Backpropagation: Perceptron

□ Now we can update the weights using the formula:

$$w^{new} = w - \alpha \frac{\partial E}{\partial w}$$

*Minus sign: we move the weights in the direction **opposite** the gradient (i.e. the direction that reduces rather than increases the error)*

The learning rate (hyperparameter). The bigger the learning rate, the greater the change to the weights.

$$w_1^{(new)} = 0.15 - 0.05 \times (-0.049) = 0.1523$$

$$w_2^{(new)} = -0.3 - 0.05 \times (-0.01) = -0.2995$$



Backpropagation: Perceptron

□ Now do the same with bias:

$$b^{(new)} = b - \alpha \frac{\partial E}{\partial b}$$

$$b^{(new)} = 0 - 0.05 \times (-0.098) = 0.005$$



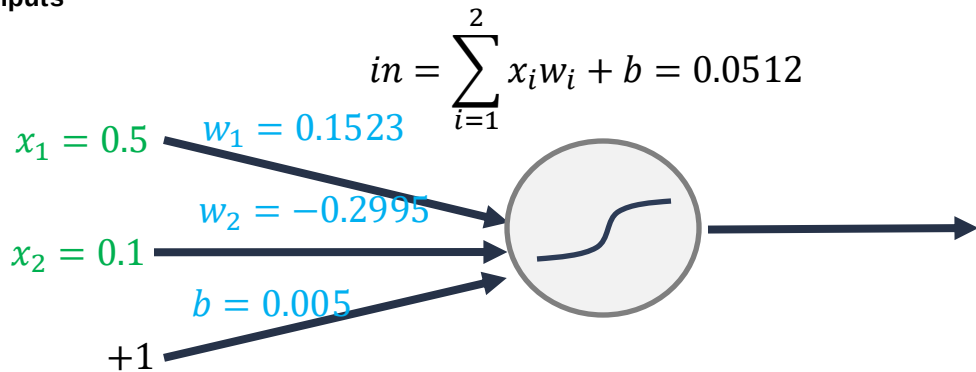
Backpropagation: Perceptron

□ Let's see if the network performs better

$$E = \frac{1}{2} (\text{target} - \text{out})^2 = \frac{1}{2} (0.9 - 0.513)^2 = 0.0749$$

(previous error: 0.0757)

inputs



Python Programming Example



Perceptron Backpropagation.ipynb

Perceptron:

Implementing Gradient Descent using
Matrix Notation

Perceptron:

Matrix Notation: Single Instance



Gradient Descent: Perceptron

Matrix notation (single instance $x = [x_1, \dots, x_k]$):

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_k \end{bmatrix}$$

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} \partial E / w_1 \\ \vdots \\ \partial E / w_k \end{bmatrix}$$



Gradient Descent: Perceptron

Matrix notation (single instance $x = [x_1, \dots, x_k]$):

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_k \end{bmatrix}$$

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} \partial E / w_1 \\ \vdots \\ \partial E / w_k \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} \frac{\partial in}{\partial w_1} \\ \vdots \\ \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} \frac{\partial in}{\partial w_k} \end{bmatrix}$$



Gradient Descent: Perceptron

Matrix notation (single instance $x = [x_1, \dots, x_k]$):

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_k \end{bmatrix}$$

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} \partial E / w_1 \\ \vdots \\ \partial E / w_k \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_1 \\ \vdots \\ \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_k \end{bmatrix}$$



Gradient Descent: Perceptron

Matrix notation (single instance $x = [x_1, \dots, x_k]$):

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_k \end{bmatrix}$$

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} \partial E / w_1 \\ \vdots \\ \partial E / w_k \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_1 \\ \vdots \\ \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_k \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in}$$



Gradient Descent: Perceptron

Matrix notation (single instance $x = [x_1, \dots, x_k]$):

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_k \end{bmatrix}$$

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} \partial E / w_1 \\ \vdots \\ \partial E / w_k \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_1 \\ \vdots \\ \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_k \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} = (x)^T \frac{\partial E}{\partial out} \frac{\partial out}{\partial in}$$



Gradient Descent: Perceptron

Matrix notation (single instance $x = [x_1, \dots, x_k]$):

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} \partial E / w_1 \\ \vdots \\ \partial E / w_k \end{bmatrix} = \begin{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_1 \\ \vdots \\ \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} x_k \end{bmatrix} = \begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in} = (x)^T \frac{\partial E}{\partial out} \frac{\partial out}{\partial in}$$

$$\frac{\partial E_x}{\partial b} = \frac{\partial E}{\partial out} \frac{\partial out}{\partial in}$$

Python Programming Example



Perceptron Backpropagation.ipynb

Perceptron:

Matrix Notation: Batch



Gradient Descent: Perceptron

Matrix notation (batch $X = (x^1 \dots x^n)$ where: $x^i = (x_1^i, \dots, x_k^i)$):

$$\frac{\partial E_x}{\partial W} = \begin{bmatrix} x_1 \\ \vdots \\ x_k \end{bmatrix} \frac{\partial E}{\partial out} \frac{\partial out}{\partial in}$$

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \sum_{i=1}^n \left(\begin{bmatrix} x_1^i \\ \vdots \\ x_k^i \end{bmatrix} \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \right)$$



Calculation for a single input x^i



Batch Gradient Descent: Perceptron

Matrix notation (batch $X = (x^1 \dots x^n)$ where: $x^i = (x_1^i, \dots, x_k^i)$):

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \sum_{i=1}^n \left(\begin{bmatrix} x_1^i \\ \vdots \\ x_k^i \end{bmatrix} \frac{\partial E(x^i)}{\partial \text{out}(x^i)} \frac{\partial \text{out}(x^i)}{\partial \text{in}(x^i)} \right) = \begin{bmatrix} \frac{1}{n} \sum_{i=1}^n x_1^i \frac{\partial E(x^i)}{\partial \text{out}(x^i)} \frac{\partial \text{out}(x^i)}{\partial \text{in}(x^i)} \\ \vdots \\ \frac{1}{n} \sum_{i=1}^n x_k^i \frac{\partial E(x^i)}{\partial \text{out}(x^i)} \frac{\partial \text{out}(x^i)}{\partial \text{in}(x^i)} \end{bmatrix}$$

$\frac{\partial E_X}{\partial w_1}$
 $\frac{\partial E_X}{\partial w_k}$



Batch Gradient Descent: Perceptron

Matrix notation (batch $X = (x^1 \dots x^n)$ where: $x^i = (x_1^i, \dots, x_k^i)$):

$$\frac{\partial E_X}{\partial W} = \begin{bmatrix} \frac{1}{n} \sum_{i=1}^n x_1^i \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \\ \vdots \\ \frac{1}{n} \sum_{i=1}^n x_k^i \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \end{bmatrix} =$$

We can write this in a more compact way...



Batch Gradient Descent: Perceptron

Matrix notation (batch $X = (x^1 \dots x^n)$ where: $x^i = (x_1^i, \dots, x_k^i)$):

$$\frac{\partial E_X}{\partial W} = \begin{bmatrix} \frac{1}{n} \sum_{i=1}^n x_1^i \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \\ \vdots \\ \frac{1}{n} \sum_{i=1}^n x_k^i \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \end{bmatrix} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right)$$

where:

$$\frac{\partial E_X}{\partial out_X} = \begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} \\ \vdots \\ \frac{\partial E(x^n)}{\partial out(x^n)} \end{bmatrix} \quad \frac{\partial out_X}{\partial in_X} = \begin{bmatrix} \frac{\partial out(x^1)}{\partial in(x^1)} \\ \vdots \\ \frac{\partial out(x^n)}{\partial in(x^n)} \end{bmatrix}$$



Batch Gradient Descent: Perceptron

Exercise - Demonstrate that:

Matrix multiplication

$$\frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right) = \begin{bmatrix} \frac{1}{n} \sum_{i=1}^n x_1^i \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \\ \vdots \\ \frac{1}{n} \sum_{i=1}^n x_k^i \frac{\partial E(x^i)}{\partial out(x^i)} \frac{\partial out(x^i)}{\partial in(x^i)} \end{bmatrix} \text{ - matrix } [k, 1]$$

Elementwise multiplication

$$\frac{\partial E_X}{\partial out_X} = \begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} \\ \vdots \\ \frac{\partial E(x^n)}{\partial out(x^n)} \end{bmatrix} \quad \frac{\partial out_X}{\partial in_X} = \begin{bmatrix} \frac{\partial out(x^1)}{\partial in(x^1)} \\ \vdots \\ \frac{\partial out(x^n)}{\partial in(x^n)} \end{bmatrix} \quad X = \begin{bmatrix} x_1^1 & \cdots & x_k^1 \\ \vdots & \ddots & \vdots \\ x_1^n & \cdots & x_k^n \end{bmatrix}$$



Batch Gradient Descent: Perceptron

Matrix notation (batch $X = (x^1 \dots x^n)$ where: $x^i = (x_1^i, \dots, x_k^i)$):

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \left(\underbrace{X^T}_{\text{matrix } [k, n]} \times \underbrace{\left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right)}_{\text{matrix } [n, 1]} \right) = \begin{bmatrix} \partial E_X / \partial w_1 \\ \vdots \\ \partial E_X / \partial w_k \end{bmatrix} - \text{matrix } ([k \times 1])$$

$$\frac{\partial E}{\partial b} = \frac{1}{n} \left(\underbrace{\vec{1}}_{\substack{\uparrow \\ \text{matrix } [1, n]}} \times \underbrace{\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X}}_{\text{matrix } [n, 1]} \right) - \text{scalar}$$



Batch Gradient Descent: Perceptron

Exercise: By doing appropriate calculations, demonstrate that for $X = \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix}$ the gradient of a perceptron $\left(\frac{\partial E_X}{\partial w}\right)$ is a matrix of dimension $[2 \times 1]$



Batch Gradient Descent: Perceptron

Exercise: By doing appropriate calculations, demonstrate that for $X = \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix}$ the gradient of a perceptron $\left(\frac{\partial E_X}{\partial W}\right)$ is a matrix of dimension $[2 \times 1]$

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right) \text{ where } n=2$$



Batch Gradient Descent: Perceptron

Exercise: By doing appropriate calculations, demonstrate that for $X = \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix}$ the gradient of a perceptron $\left(\frac{\partial E}{\partial W}\right)$ is a matrix of dimension $[2 \times 1]$

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right) \text{ where } n=2$$

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right) = \frac{1}{2} \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix} \times \left(\begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} \\ \frac{\partial E(x^2)}{\partial out(x^2)} \end{bmatrix} * \begin{bmatrix} \frac{\partial out(x^1)}{\partial in(x^1)} \\ \frac{\partial out(x^2)}{\partial in(x^2)} \end{bmatrix} \right)$$



Batch Gradient Descent: Perceptron

Exercise: By doing appropriate calculations, demonstrate that for $X = \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix}$ the gradient of a perceptron $\left(\frac{\partial E}{\partial W}\right)$ is a matrix of dimension $[2 \times 1]$

$$\frac{\partial E}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E}{\partial out} * \frac{\partial out}{\partial in} \right) \right) \text{ where } n=2$$

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right) = \frac{1}{2} \begin{bmatrix} x_1^1 & x_1^2 \\ x_2^1 & x_2^2 \end{bmatrix} \times \left(\begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} \\ \frac{\partial E(x^2)}{\partial out(x^2)} \end{bmatrix} * \begin{bmatrix} \frac{\partial out(x^1)}{\partial in(x^1)} \\ \frac{\partial out(x^2)}{\partial in(x^2)} \end{bmatrix} \right) = \frac{1}{2} \begin{bmatrix} x_1^1 & x_1^2 \\ x_2^1 & x_2^2 \end{bmatrix} \times$$

$$\begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} * \frac{\partial out(x^1)}{\partial in(x^1)} \\ \frac{\partial E(x^2)}{\partial out(x^2)} * \frac{\partial out(x^2)}{\partial in(x^2)} \end{bmatrix}$$



Batch Gradient Descent: Perceptron

Exercise: By doing appropriate calculations, demonstrate that for $X = \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix}$ the gradient of a perceptron $\left(\frac{\partial E}{\partial W}\right)$ is a matrix of dimension $[2 \times 1]$

$$\frac{\partial E}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E}{\partial out} * \frac{\partial out}{\partial in} \right) \right) \text{ where } n=2$$

$$\frac{\partial E_X}{\partial W} = \frac{1}{n} \left(X^T \times \left(\frac{\partial E_X}{\partial out_X} * \frac{\partial out_X}{\partial in_X} \right) \right) = \frac{1}{2} \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix} \times \left(\begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} \\ \frac{\partial E(x^2)}{\partial out(x^2)} \end{bmatrix} * \begin{bmatrix} \frac{\partial out(x^1)}{\partial in(x^1)} \\ \frac{\partial out(x^2)}{\partial in(x^2)} \end{bmatrix} \right) = \frac{1}{2} \begin{bmatrix} x_1^1 & x_2^1 \\ x_1^2 & x_2^2 \end{bmatrix} \times$$

$$\begin{bmatrix} \frac{\partial E(x^1)}{\partial out(x^1)} * \frac{\partial out(x^1)}{\partial in(x^1)} \\ \frac{\partial E(x^2)}{\partial out(x^2)} * \frac{\partial out(x^2)}{\partial in(x^2)} \end{bmatrix} =$$

$$\begin{bmatrix} \frac{1}{2} \left(\frac{\partial E(x^1)}{\partial out(x^1)} * \frac{\partial out(x^1)}{\partial in(x^1)} \right) & \frac{1}{2} \left(\frac{\partial E(x^2)}{\partial out(x^2)} * \frac{\partial out(x^2)}{\partial in(x^2)} \right) \end{bmatrix}$$

Python Programming Example



Perceptron Backpropagation/Batch Gradient Descent.ipynb

Coming Next:

More on Gradient Descent